

Trabalho Prático 1

Exercício 2: Sudoku Genérico como CSP

Um Sudoku clássico consiste numa grelha de dimensão $n^2 \times n^2$, onde cada linha, coluna e cada bloco de dimensão $n \times n$ contém valores entre 1 e n^2 , sem repetições.

O objetivo deste exercício consiste na implementação de um gerador e resolvidor genérico de Sudoku através de um modelo de satisfação de restrições (CSP).

A implementação foi feita em função de n , permitindo utilizar o mesmo código para diferentes dimensões. Por exemplo, $n = 3$ corresponde ao Sudoku clássico 9×9 , enquanto $n = 2$ corresponde a uma grelha 4×4 .

A ideia principal da resolução consiste em representar linhas, colunas, blocos e pistas através de grupos de células. Como todos estes grupos seguem a mesma regra de valores diferentes, podem ser tratados da mesma forma pelo modelo CSP.

```
In [ ]: import marimo as mo
import random
from ortools.sat.python import cp_model
```

R1: box

A classe `box` representa um conjunto genérico de células da grelha.

Cada célula é identificada pelas coordenadas `(linha, coluna)` e pode ter um valor fixo pertencente ao intervalo $[1, n^2]$ (célula fixa a esse valor) ou o valor `None` (célula livre).

As células são guardadas num dicionário com a forma `(linha, coluna) -> valor`.

A classe `box` não contém qualquer conhecimento específico sobre linhas, colunas ou blocos.

O método `add()` permite adicionar novas células ao grupo, rejeitando coordenadas exteriores à grelha $n^2 \times n^2$ (é lançada uma exceção `IndexError`) e valores que não pertencem ao domínio (é lançada uma exceção `ValueError`).

A classe disponibiliza ainda uma representação matricial $n^2 \times n^2$, para facilitar a visualização, na qual as células livres são representadas por 0.

```

In [ ]: # função que verifica as coordenadas e valor das células
def valida_celula(i,j,val,dim):
    # verifica se as coordenadas são válidas
    if not (0<=i<dim and 0<=j<dim):
        raise IndexError("coordenadas inválidas")

    # verifica se o valor é válido
    if not (val is None or 0<val<=dim):
        raise ValueError("valor inválido")

# função que imprime a matriz
def imprime_matriz(m):
    for l in m:
        print(l)

    print()

# grupo genérico de células
class box:

    # inicializa um box
    def __init__(self, n, cells = None):
        self.n = n
        self.dim = n**2
        self.cells = {}

    # aceita células iniciais
    if not (cells is None):
        for (i,j),val in cells.items():
            valida_celula(i,j,val,self.dim)
            self.cells[(i,j)] = val

    # adiciona uma célula ao box
    def add(self, i, j, val=None):

        # verifica se a célula é válida
        valida_celula(i,j,val,self.dim)

        self.cells[(i,j)] = val

    # matriz de box
    def matrix(self):

        m = []

        # reseta a matriz com 0
        for i in range(self.dim):
            linha = []
            for j in range(self.dim):
                linha.append(0)

```

```

        m.append(linha)

    # atualiza as células fixas
    for (i,j),val in self.cells.items():
        if val != None:
            m[i][j] = val

    return m

```

Teste sobre as funcionalidades de R1:

```

In [ ]: # cria um box 4x4
def teste_box_n2():
    print("n = 2")
    b = box(2)
    b.add(0,1,3) # posição (0,1) com valor 3
    b.add(1,2) # posição (1,2) com valor None
    b.add(3,3,4)
    # imprime a matriz b para verificar se foi adicionado com sucesso
    imprime_matriz(b.matrix())

# cria um box 9x9 com células iniciais
def teste_box_n3():
    print("n = 3")
    b = box(3, {(4,8): 9,
                (2,4): 1,
                (6,1): None,
                (7,2): 6,
                (1,5): 3,
                (3,3): None})
    imprime_matriz(b.matrix())

teste_box_n2()
print()
teste_box_n3()

# verifica se age corretamente com erros
def teste_erros_box():
    try:
        b = box(2, {(4,8): 9})
        print("Erro: aceitou coordenadas inválidas")

    # deve detetar as coordenadas inválidas e lançar exceção
    except IndexError:
        print("Rejeitou coordenadas inválidas")

    try:
        b = box(2, {(1,2): 9})
        print("Erro: aceitou valor inválido")

    # deve detetar o valor inválido e lançar exceção
    except ValueError:

```

```

        print("Rejeitou valor inválido")

teste_erros_box()

```

```

n = 2
[0, 3, 0, 0]
[0, 0, 0, 0]
[0, 0, 0, 0]
[0, 0, 0, 4]

```

```

n = 3
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 3, 0, 0, 0]
[0, 0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 9]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 6, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 0, 0]

```

Rejeitou coordenadas inválidas
Rejeitou valor inválido

Os testes confirmam que a classe `box` pode ser criada tanto vazia como com um conjunto inicial de células. As células fornecidas inicialmente são corretamente armazenadas e representadas na matriz, sendo igualmente verificadas as situações de coordenadas e valores inválidos.

R2: cube

A classe `cube` representa um bloco $n \times n$ do Sudoku. Sendo por raiz um conjunto de células, `cube` foi definida como uma subclasse de `box`.

Cada bloco é identificado pelos índices (i, j) , com $0 \leq i, j < n$. A posição inicial do bloco é dada por $(i \cdot n, j \cdot n)$.

A partir dessa posição são percorridas `n` linhas e `n` colunas, sendo cada célula adicionada ao grupo através do método `add()` herdado de `box`.

```

In [ ]: # um bloco n*n do Sudoku
class cube(box): # subclasse de box

    # inicializa um cube
    def __init__(self, n, i, j):
        super().__init__(n) # herda de box os dados iniciais

    # verifica se i e j são válidos
    valida_celula(i, j, None, n)

```

```

# calcula o inicio e fim
linha_inicial = i * n
coluna_inicial = j * n

# adiciona com self.add()
for l in range(n):
    for c in range(n):
        self.add(linha_inicial + l,
                  coluna_inicial + c)

```

Teste sobre as funcionalidades de R2:

```

In [ ]: # cria um cube com n=2
def teste_cube_n2():
    print("n = 2")
    c = cube(2,0,1) # linha inicial=0 e coluna inicial=2
    print(c.cells)

# cria um cube com n=3
def teste_cube_n3():
    print("n = 3")
    c = cube(3,1,2) # linha inicial=3 e coluna inicial=6
    print(c.cells)

teste_cube_n2()
print()
teste_cube_n3()

# verifica se age corretamente com erros
def teste_erros_cube():
    try:
        c = cube(3,4,0) # bloco de n=3, linha inicial=12 (fora do bloco)
        print("Erro: aceitou posição inválida")

        # deve detetar posição inválida e lançar exceção
    except IndexError:
        print("Rejeitou posição inválida")

print()
teste_erros_cube()

```

n = 2

```
{(0, 2): None, (0, 3): None, (1, 2): None, (1, 3): None}
```

n = 3

```
{(3, 6): None, (3, 7): None, (3, 8): None, (4, 6): None, (4, 7): None, (4, 8): None, (5, 6): None, (5, 7): None, (5, 8): None}
```

Rejeitou posição inválida

Os testes realizados confirmam que a classe `cube` identifica corretamente as células pertencentes a um bloco $n^2 \times n^2$, tanto para `n = 2` como para `n = 3`.

Verifica-se também que a classe rejeita índices de bloco inválidos, através da geração de uma exceção apropriada.

Assim, conclui-se que a implementação de `cube` cumpre os requisitos definidos para a representação e validação dos blocos do Sudoku.

R3: path

A classe `path` representa uma sequência reta de células entre duas coordenadas da grelha, podendo essa sequência ser horizontal ou vertical.

Tal como `cube`, a classe `path` é definida como uma subclasse de `box`. Esta escolha permite reutilizar a estrutura de `box` e reutilizar o seu método `add()` para adicionar células. Assim, `path` fica responsável apenas por determinar quais as células que pertencem ao percurso definido entre as coordenadas inicial e final.

Dadas duas coordenadas `inicio` e `fim`, é verificado 3 casos:

- se as linhas forem iguais, o percurso é horizontal;
- se as colunas forem iguais, o percurso é vertical;
- caso contrário, as coordenadas não definem um percurso reto válido.

O percurso deve ainda funcionar nos dois sentidos. Por exemplo, `path(3, (2,1), (2,5))` e `path(3, (2,5), (2,1))` representam o mesmo conjunto de células, apenas percorrido em sentidos opostos.

Para controlar o sentido do percurso, é utilizado um `passo` de valor `1` quando o índice aumenta e `-1` quando diminui.

Desta forma, a mesma classe pode ser utilizada posteriormente para representar tanto as linhas como as colunas do Sudoku.

```
In [ ]: # Apoio LLM: explicação sobre a utilização de passo 1/-1 para permitir
        percorrer o path nos dois sentidos

        # uma sequência reta de células
        class path(box):

            # inicializa um path
            def __init__(self,n,inicio,fim):
                super().__init__(n)

            # saber as coordenadas e verificar se formam uma reta
            l_inicio, c_inicio = inicio
            l_fim, c_fim = fim
```

```

# verifica se estão dentro da grelha
if not(0 <= l_inicio < self.dim and
      0 <= c_inicio < self.dim and
      0 <= l_fim < self.dim and
      0 <= c_fim < self.dim):
    raise IndexError("coordenadas inválidas")

# descobre se é horizontal ou vertical
if l_inicio == l_fim: # mesma linha
    if c_inicio <= c_fim: # aumenta
        passo = 1
    else: # diminui
        passo = -1

    for j in range(c_inicio, c_fim+passo, passo):
        self.add(l_inicio, j)

elif c_inicio == c_fim:
    if l_inicio <= l_fim: # aumenta
        passo = 1
    else: # diminui
        passo = -1

    for i in range(l_inicio, l_fim+passo, passo):
        self.add(i, c_inicio)

else:
    raise ValueError("coordenadas não formam uma reta")

```

Teste sobre as funcionalidades de R3:

```

In [ ]: # cria um path com n = 3
def teste_path():
    print("n = 3")
    p = path(3, (2,1), (2,5)) # início em (2,1) e fim em (2,5)
    print(p.cells)

# cria o path no sentido contrário
def teste_path_contrario():
    p = path(3, (2,5), (2,1))
    print(p.cells)

teste_path()
print()
teste_path_contrario()

# verifica se age corretamente com erros
def teste_erros_path():
    try:
        p = path(3, (1,2), (2,7))
        print("Erro: aceitou coordenadas não retas")

```

```
# deve detetar posição inválida e lançar exceção
except ValueError:
    print("Rejeitou coordenadas não retas")

print()
teste_erros_path()
```

n = 3

{(2, 1): None, (2, 2): None, (2, 3): None, (2, 4): None, (2, 5): None}

{(2, 5): None, (2, 4): None, (2, 3): None, (2, 2): None, (2, 1): None}

Rejeitou coordenadas não retas

Os testes confirmam que a classe `path` consegue representar corretamente uma sequência reta de células entre duas coordenadas, funcionando tanto no sentido direto como no sentido inverso.

Foi também verificado que coordenadas que não pertencem à mesma linha nem à mesma coluna são corretamente rejeitadas.

Desta forma, conclui-se que `path` pode ser utilizado de forma segura para representar linhas e colunas do Sudoku em ambos os sentidos.

R4: geração aleatória de pistas

As pistas iniciais são representadas por um `box`, uma vez que correspondem apenas a células com valores fixos.

Para gerar pistas foi criada a função `pistas_aleatorias(n, k=None)`. Caso `k` não seja indicado, é utilizado por omissão o valor de `n`.

A função gera aleatoriamente:

- uma linha;
- uma coluna;
- um valor entre `1` e `n2`.

A escolha aleatória das posições e dos valores é realizada através da biblioteca `random` do Python.

Antes de adicionar uma nova pista é verificado se a posição já foi utilizada, evitando que duas pistas ocupem a mesma célula.

É ainda verificado se `k` é válido, não sendo permitido pedir um número de pistas superior ao número total de células da grelha.

No final, a função devolve o `box` contendo todas as pistas geradas.

```
In [ ]: # Apoio LLM: discussão sobre geração de posições aleatórias sem repetir células

def pistas_aleatorias(n, k=None):

    pistas = box(n) # cria um box associado à grelha n² x n²

    # quando k é None, considerar k = n
    if k is None:
        k = n

    # verifica se k é válido
    if k < 0 or k > pistas.dim**2: # for negativo ou maior que o total de células da grelha
        raise ValueError("número de pistas inválido")

    # posição aleatória -> valor aleatório
    while len(pistas.cells) < k:

        # gerar uma posição aleatória
        linha = random.randint(0, pistas.dim - 1)
        coluna = random.randint(0, pistas.dim - 1)

        # verificar que esta posição gerada é vazia
        if (linha,coluna) not in pistas.cells:
```

```

        # gerar o valor fixo para ela
        valor = random.randint(1, pistas.dim)
        # adicionar à pista
        pistas.add(linha, coluna, valor)

    return pistas

```

Teste sobre as funcionalidades de R4:

```

In [ ]: def teste_pistas():
        p = pistas_aleatorias(2, 3) # n = 2, k = 3
        print(p.cells)
        print("pistas geradas: " + str(len(p.cells)))
        m = p.matrix()
        imprime_matriz(m)

teste_pistas()

# verifica se age corretamente com erros
def teste_erros_pistas():
    try:
        p = pistas_aleatorias(3, -1)
        print("Erro: aceitou número de pistas inválido")

        # deve detetar posição inválida e lançar exceção
    except ValueError:
        print("Rejeitou número de pistas inválido")

print()
teste_erros_pistas()

```

```

{(3, 2): 4, (2, 3): 2, (1, 3): 4}
pistas geradas: 3
[0, 0, 0, 0]
[0, 0, 0, 4]
[0, 0, 0, 2]
[0, 0, 4, 0]

```

Rejeitou número de pistas inválido

Os testes realizados confirmam que a função produz exatamente o número de pistas solicitado, sem repetir posições e atribuindo apenas valores válidos para a dimensão da grelha.

A representação matricial permite ainda verificar visualmente a localização das pistas geradas.

Foi também testado um valor inválido para `k`, sendo corretamente rejeitado através de uma exceção.

Assim, conclui-se que a geração aleatória de pistas cumpre os requisitos definidos no R4.

R5: criação do modelo CSP

Nesta fase é construído o modelo de satisfação de restrições responsável pela resolução do Sudoku.

Foi utilizada a biblioteca `Or-Tools`, através de `CpModel()`.

A função `modelo_sudoku(n)` cria uma variável inteira para cada célula da grelha. Como a dimensão do Sudoku é $n^2 \times n^2$, cada uma pode assumir valores entre `1` e `n2`.

As variáveis são guardadas através das respetivas coordenadas, onde `variaveis[i][j]` representa o valor atribuído à célula localizada na linha `i` e coluna `j`.

Foi depois criada a função `add_grupos`, que recebe os grupos de células e para cada grupo são recolhidas as variáveis correspondentes e é aplicada a restrição `AddAllDifferent()`. Assim, todas as células pertencentes ao mesmo grupo devem possuir valores diferentes. Caso alguma célula tenha um valor fixo, é também adicionada uma restrição que obriga a respetiva variável a assumir esse valor.

Como todos os grupos (`box`, `cube`, `path` e `pistas`) mantêm a mesma estrutura `cells`, o modelo não necessita de distinguir se um grupo corresponde a uma linha, coluna, bloco ou conjunto de pistas, mantendo a modelação genérica pretendida.

A função `resolve_sudoku` cria um `CpSolver` e tenta resolver o modelo. Quando existe uma solução, os valores obtidos são guardados numa matriz. Caso não exista solução, a função devolve `None`.

A implementação do modelo segue exatamente esta separação entre criação das variáveis, adição dos grupos e resolução.

```
In [ ]: # Apoio LLM: explicação inicial do funcionamento de CpModel,
        AddAllDifferent e CpSolver

# cria o modelo CSP
def modelo_sudoku(n):
    dim = n**2 # define a dimensão da grelha do Sudoku

    # cria o modelo CP_SAT (sugerido no enunciado)
    model = cp_model.CpModel()

    # dicionário que guarda as variáveis da grelha
    vars= {}

    # cria as variáveis de todas as células
```

```

    for i in range(dim):
        vars[i] = {}
        for j in range(dim):
            vars[i][j] = model.NewIntVar(1, dim, f"x_{i}_{j}") # cada
            variável assume valores entre 1 e n²

    return model, vars

# adiciona ao modelo as restrições associadas a grupos
def add_grupos(model, variaveis, grupos):

    # como os diferentes grupos de células mantém a mesma estrutura base:
    box, é possível tratar box, cube, path e pistas da mesma maneira

    # para cada grupo
    for grupo in grupos:

        # obtém as variáveis correspondentes
        temp = []
        for (i,j) in grupo.cells:
            temp.append(variaveis[i][j])

        # aplica a restrição AllDifferent
        model.AddAllDifferent(temp)

        # trata das células fixas
        for (i,j), val in grupo.cells.items():
            if val is not None:
                model.Add(variaveis[i][j] == val)

# resolve o modelo
def resolve_sudoku(model, variaveis, dim):

    # cria o solver e resolve o modelo
    solver = cp_model.CpSolver()
    status = solver.Solve(model)

    # se houver solução
    if status == cp_model.OPTIMAL or status == cp_model.FEASIBLE:

        # cria a matriz de solução
        matriz_solucão = []

        for i in range(dim):
            linha = []
            for j in range(dim):
                valor = solver.Value(variaveis[i][j])
                linha.append(valor)

            matriz_solucão.append(linha)

        return matriz_solucão

    # não existe solução

```

```

    return None

# imprime a matriz de solução
def imprime_sudoku(m, n):
    dim = n**2

    # não houve solução
    if m is None:
        print("sudoku sem solução")
        return

    for i in range(dim):
        if i % n == 0 and i != 0: # separação horizontal entre blocos
            print("-" * (dim * 2 + n - 1))

        for j in range(dim):
            if j % n == 0 and j != 0: # separação vertical entre blocos
                print("|", end=" ")

            print(m[i][j], end=" ")

        print()

```

Teste sobre as funcionalidades de R5:

```

In [ ]: # cria um modelo para um Sudoku nxn
def teste_R5(n):
    print("n =", n)

    dim = n**2

    model, vars = modelo_sudoku(n)

    # grupo correspondente à primeira linha
    linha = path(n, (0,0), (0,dim-1))

    # grupo contendo uma célula fixa
    pistas = box(n, {(0,0): 1})

    # adicionar simultaneamente as restrições dos dois grupos
    add_grupos(model, vars, [linha, pistas])

    # resolver
    m = resolve_sudoku(model, vars, dim)
    imprime_sudoku(m, n)

teste_R5(2)
print()
teste_R5(3)

```

```

n = 2
1 4 | 3 2
1 1 | 1 1
-----
1 1 | 1 1
1 1 | 1 1

n = 3
1 9 8 | 7 6 5 | 4 3 2
1 1 1 | 1 1 1 | 1 1 1
1 1 1 | 1 1 1 | 1 1 1
-----
1 1 1 | 1 1 1 | 1 1 1
1 1 1 | 1 1 1 | 1 1 1
1 1 1 | 1 1 1 | 1 1 1
-----
1 1 1 | 1 1 1 | 1 1 1
1 1 1 | 1 1 1 | 1 1 1
1 1 1 | 1 1 1 | 1 1 1

```

Foi criado um modelo simples contendo uma linha representada através de `path` e uma célula fixa representada através de `box`.

A solução obtida apresenta valores distintos em toda a primeira linha e mantém a célula `(0,0)` fixada no valor `1`, confirmando tanto a aplicação de `AddAllDifferent()` como das restrições de valores previamente definidos.

R6: montagem e resolução do Sudoku completo

Nesta fase são reunidas todas as estruturas e restrições definidas anteriormente, permitindo construir o modelo completo do Sudoku.

A função `sudoku(n, k=None)` começa por criar o modelo CSP e as variáveis.

De seguida, são adicionados ao mesmo modelo os diferentes grupos que definem as regras do Sudoku:

- cada linha é representada por um `path` horizontal;
- cada coluna é representada por um `path` vertical;
- cada bloco $n \times n$ é representado por um `cube`;
- um grupo de pistas aleatórias, utilizando `box`.

Todos estes grupos são guardados numa lista e enviados de uma só vez para `add_grupos`, que aplica a restrição `AddAllDifferent()` e fixa os valores previamente atribuídos, quando existirem.

Após a adição de todas as restrições, o modelo é resolvido através de `CpSolver`. Caso seja encontrada uma solução, os valores atribuídos às variáveis são apresentados

em formato matricial. Caso contrário, é indicado que não foi encontrada uma solução.

Desta forma, a resolução do Sudoku resulta da combinação das várias estruturas genéricas desenvolvidas nas etapas anteriores, sem ser necessário implementar separadamente a lógica das linhas, colunas, blocos e pistas.

```
In [ ]: # criar modelo
def sudoku(n, k=None): # k opcional para gerar pistas iniciais
    dim = n**2

    # criar modelo
    model, vars = modelo_sudoku(n)

    # criar listas de grupos
    grupos = []

    # adiciona todas as linhas com path
    for i in range(dim):
        # cada linha vai da coluna 0 até coluna dim-1
        linha = path(n, (i,0), (i,dim-1))
        grupos.append(linha)

    # adiciona todas as colunas com path
    for j in range(dim):
        # cada coluna vai da linha 0 até linha dim-1
        coluna = path(n, (0,j), (dim-1,j))
        grupos.append(coluna)

    # adiciona todos os blocos com cube
    # existem n x n blocos
    for i in range(n):
        for j in range(n):
            bloco = cube(n,i,j)
            grupos.append(bloco)

    # adiciona pistas aleatórias
    pistas = pistas_aleatorias(n, k)
    grupos.append(pistas)

    # passar essa lista ao add_grupos
    add_grupos(model, vars, grupos)

    # resolver
    res = resolve_sudoku(model, vars, n**2)

    # imprimir solução
    imprime_sudoku(res, n)

    return res, pistas
```

Teste sobre as funcionalidades de R6:

```
In [ ]: sudoku_n2 = sudoku(2)
```

```

2 4 | 3 1
3 1 | 2 4
-----
1 3 | 4 2
4 2 | 1 3

```

```
In [ ]: sudoku_n3 = sudoku(3)
```

```

3 1 4 | 8 7 2 | 5 6 9
9 7 5 | 1 6 3 | 4 2 8
2 8 6 | 9 5 4 | 1 7 3
-----
4 9 1 | 3 8 6 | 7 5 2
8 5 2 | 7 9 1 | 6 3 4
6 3 7 | 4 2 5 | 9 8 1
-----
7 6 9 | 2 1 8 | 3 4 5
1 4 8 | 5 3 7 | 2 9 6
5 2 3 | 6 4 9 | 8 1 7

```

Depois de construídas todas as linhas, colunas e blocos, estes são reunidos numa lista juntamente com o grupo de pistas.

Todos os grupos são então fornecidos à função `add_grupos`, que aplica ao modelo as restrições correspondentes sem necessidade de distinguir a origem de cada grupo.

Esta abordagem evidencia a generalidade da estrutura desenvolvida: linhas, colunas, blocos e pistas são tratados uniformemente pelo modelo CSP.

Validação final da solução

Além de obter uma solução através do solver, foi feita uma validação automática para confirmar que a grelha encontrada corresponde realmente a um Sudoku válido.

Ou seja, a solução tem de respeitar as seguintes regras:

1. cada linha contém exatamente os valores de `1` a `n2`, sem repetições;
2. cada coluna contém exatamente os valores de `1` a `n2`, sem repetições;
3. cada bloco $n \times n$ contém exatamente os valores de `1` a `n2`;
4. todas as pistas mantêm os valores que lhes foram inicialmente atribuídos.

Para isso, foi criada uma lista com os valores esperados `[1, 2, ..., n2]`.

Cada linha/coluna da solução é ordenada e comparada com esta lista.

Para os blocos, é calculada a posição inicial de cada bloco e são recolhidas as suas $n \times n$ células. Os valores obtidos são novamente comparados com a lista esperada.

As pistas são verificadas separadamente. Para cada posição existente no `box` das pistas é confirmado que a solução mantém exatamente o valor que tinha sido

inicialmente atribuído.

```
In [ ]: # Apoio LLM: revisão da estratégia usada para validar linhas, colunas,
        blocos e pistas

def valida_linhas(matriz, lista_comparacao, dim):
    for i in range(dim):
        linha = []

        for j in range(dim):
            # recolhe os valores da solução
            linha.append(matriz[i][j])

        # ordena os valores e compara com a lista esperada
        if sorted(linha) != lista_comparacao:
            return False

    return True

def valida_colunas(matriz, lista_comparacao, dim):
    for j in range(dim):
        coluna = []

        for i in range(dim):
            # recolhe os valores da solução
            coluna.append(matriz[i][j])

        # ordena os valores e compara com a lista esperada
        if sorted(coluna) != lista_comparacao:
            return False

    return True

def valida_blocos(matriz, lista_comparacao, n):
    for bloco_i in range(n):
        for bloco_j in range(n):
            bloco = []

            # calcula inicio do bloco
            linha_inicial = bloco_i * n
            coluna_inicial = bloco_j * n

            for l in range(n):
                for c in range(n):
                    bloco.append(matriz[linha_inicial + l][coluna_inicial +
c])

            # ordena os valores e compara com a lista esperada
            if sorted(bloco) != lista_comparacao:
                return False

    return True
```

```

def valida_pistas(matriz, pistas):
    # para cada variável da pista
    for (i,j), val in pistas.cells.items():
        # verifica se o solver manteve o valor atribuido
        if matriz[i][j] != val:
            return False

    return True

def valida_solucao(matriz, pistas, n):
    dim = n**2

    # lista de valores que cada linha, coluna e bloco deve conter
    lista_esperada = list(range(1, dim+1))

    # validação das linhas
    if not valida_linhas(matriz, lista_esperada, dim):
        return False

    # validação das colunas
    if not valida_colunas(matriz, lista_esperada, dim):
        return False

    # validação dos blocos
    if not valida_blocos(matriz, lista_esperada, n):
        return False

    # validação das pistas
    if not valida_pistas(matriz, pistas):
        return False

    return True

```

```

In [ ]: matriz_n2, pistas_n2 = sudoku_n2
print("Solução válida: ", valida_solucao(matriz_n2, pistas_n2, 2))

print()

matriz_n3, pistas_n3 = sudoku_n3
print("Solução válida: ", valida_solucao(matriz_n3, pistas_n3, 3))

```

Solução válida: True

Solução válida: True

O teste final devolveu `True`, confirmando que as soluções encontradas pelo solver respeita todas as restrições verificadas: linhas, colunas, blocos e pistas.

Desta forma, fica validado automaticamente o correto funcionamento do modelo para o Sudoku resolvido.

Uso de ferramentas LLM

Durante a realização do trabalho foi utilizado o `ChatGPT` como ferramenta de apoio à resolução e revisão do código.

O uso da ferramenta ocorreu principalmente nas seguintes partes:

- interpretação inicial dos requisitos R1 a R6;
- discussão sobre a estrutura das classes `box`, `cube` e `path`;
- esclarecimento sobre a forma de percorrer um `path` nos dois sentidos;
- apoio na escolha da forma de gerar posições e valores aleatórios para as pistas;
- explicação do funcionamento básico de `CpModel`, `AddAllDifferent()` e `CpSolver`;
- revisão de erros encontrados durante os testes, como índices inválidos, construção da matriz de solução e geração dos blocos;
- apoio na definição da validação automática das linhas, colunas, blocos e pistas.

Na maior parte dos casos, o código foi desenvolvido de forma incremental: era apresentada uma tentativa de implementação, seguida da análise dos erros ou dúvidas existentes e da respetiva correção.

O diálogo utilizado durante a resolução encontra-se disponível em:

[<https://chatgpt.com/share/6ac55b45-de20-83eb-ade2-1ae8a31d6a7b>]

Conclusão da implementação base

Ao longo deste trabalho foi desenvolvido um resolvidor genérico de Sudoku recorrendo à modelação por CSP.

A solução foi construída de forma modular, partindo da classe genérica `box`, utilizada para representar qualquer grupo de células sujeito a restrições. A partir dessa estrutura foram definidas as classes `cube` e `path`, responsáveis por representar, respetivamente, os blocos e os percursos horizontais ou verticais da grelha.

Foi também implementada a geração aleatória de pistas, mantendo a mesma representação genérica através de `box`.

Na fase de modelação, foi criado um modelo CP-SAT com uma variável inteira por célula, sendo aplicadas as restrições `AllDifferent()` aos diferentes grupos e fixados os valores das pistas sempre que necessário.

A montagem final do Sudoku foi realizada através da combinação das linhas, colunas, blocos e pistas, permitindo ao solver encontrar uma solução completa para diferentes valores de `n`.

Os testes realizados com `n=2` e `n=3`, juntamente com a validação automática das linhas, colunas, blocos e pistas, confirmam que a implementação satisfaz os requisitos definidos para a parte obrigatória do trabalho.

A utilização de uma representação genérica para os grupos de células permite ainda estender facilmente o modelo a outras variantes de Sudoku, sem alterar a lógica principal de resolução.

Extensões opcionais

Sudoku diagonal (X-Sudoku)

A primeira extensão implementada foi o Sudoku diagonal, também denominado X-Sudoku.

Além das restrições normais das linhas, colunas e blocos, esta variante exige que as duas diagonais principais também contenham todos os valores de `1` a `n2` sem repetições.

Cada diagonal pode ser representada através de um `box`, adicionando as respetivas coordenadas ao grupo.

Os dois novos grupos são posteriormente fornecidos à mesma função `add_grupos` utilizada no Sudoku normal.

Desta forma, o modelo CSP não necessita de qualquer alteração, uma vez que continua apenas a receber grupos de células sujeitos à restrição

`AddAllDifferent()`.

Foi ainda criada uma validação específica das duas diagonais, comparando os valores encontrados com a lista $[1, \dots, n^2]$.

```
In [ ]: # Apoio LLM: ideia de representar as diagonais como grupos box e correção
        da coordenada final da coluna

        #X-Sudoku

        # criar os grupos de diagonais
        def diagonais(n):
            dim = n**2

            diagonal_principal = box(n)
            diagonal_secundaria = box(n)

            for i in range(dim):
                diagonal_principal.add(i, i) # posições (0,0), (1,1), (2,2), ...
                diagonal_secundaria.add(i, dim - 1 - i) # posições (0,dim-1),
```

```
(1,dim-2), (2,dim-3), ...
```

```
    return diagonal_principal, diagonal_secundaria
```

```
def sudoku_diagonal(n, k=None): # k opcional para gerar pistas iniciais  
    dim = n**2
```

```
    # criar modelo
```

```
    model, vars = modelo_sudoku(n)
```

```
    # criar listas de grupos
```

```
    grupos = []
```

```
    # adiciona todas as linhas com path
```

```
    for i in range(dim):
```

```
        # cada linha vai da coluna 0 até coluna dim-1
```

```
        linha = path(n, (i,0), (i,dim-1))
```

```
        grupos.append(linha)
```

```
    # adiciona todas as colunas com path
```

```
    for j in range(dim):
```

```
        # cada coluna vai da linha 0 até linha dim-1
```

```
        coluna = path(n, (0,j), (dim-1,j))
```

```
        grupos.append(coluna)
```

```
    # adiciona todos os blocos com cube
```

```
    # existem n x n blocos
```

```
    for i in range(n):
```

```
        for j in range(n):
```

```
            bloco = cube(n,i,j)
```

```
            grupos.append(bloco)
```

```
    # adiciona as diagonais
```

```
    dp, ds = diagonais(n)
```

```
    grupos.append(dp)
```

```
    grupos.append(ds)
```

```
    # adiciona pistas aleatórias
```

```
    pistas = pistas_aleatorias(n, k)
```

```
    grupos.append(pistas)
```

```
    # passar essa lista ao add_grupos
```

```
    add_grupos(model, vars, grupos)
```

```
    # resolver
```

```
    res = resolve_sudoku(model, vars, n**2)
```

```
    # imprimir solução
```

```
    imprime_sudoku(res, n)
```

```
    return res, pistas
```

```
# verifica se as diagonais cumprem os requisitos
```

```

def valida_diagonais(matriz, lista_esperada, dim):
    principal = []
    secundaria = []

    for i in range(dim):
        principal.append(matriz[i][i])
        secundaria.append(matriz[i][dim-1-i])

    if sorted(principal) != lista_esperada:
        return False

    if sorted(secundaria) != lista_esperada:
        return False

    return True

def valida_solucao_diagonal(matriz, pistas, n):
    dim = n**2

    # lista de valores que cada linha, coluna e bloco deve conter
    lista_esperada = list(range(1, dim+1))

    # validação do sudoku normal
    if not valida_solucao(matriz, pistas, n):
        return False

    # validação dos diagonais
    if not valida_diagonais(matriz, lista_esperada, dim):
        return False

    # validação das pistas
    if not valida_pistas(matriz, pistas):
        return False

    return True

```

Teste sobre as funcionalidades de X-Sudoku e validação:

```

In [ ]: def teste_x_sudoku(n):
        matriz, pistas = sudoku_diagonal(n)
        print("Solução válida: ", valida_solucao_diagonal(matriz, pistas, n))

        teste_x_sudoku(2)
        print()
        teste_x_sudoku(3)

```

```

2 1 | 4 3
3 4 | 1 2
-----
1 2 | 3 4
4 3 | 2 1
Solução válida: True

```

```

7 8 9 | 5 4 1 | 2 3 6
1 2 3 | 6 9 7 | 8 4 5
6 5 4 | 2 3 8 | 1 7 9
-----
2 6 7 | 3 1 5 | 4 9 8
3 1 5 | 4 8 9 | 7 6 2
4 9 8 | 7 2 6 | 5 1 3
-----
5 4 2 | 1 6 3 | 9 8 7
8 3 1 | 9 7 2 | 6 5 4
9 7 6 | 8 5 4 | 3 2 1
Solução válida: True

```

Sudoku irregular (Jigsaw)

Nesta extensão foram substituídos os blocos regulares $n \times n$ por regiões de forma arbitrária.

Cada região continua a conter exatamente n^2 células e é representada através de um objeto `box`. Como consequência, o modelo CSP não necessita de ser alterado, uma vez que cada região continua a ser tratada apenas como um grupo de células sujeito à restrição `AddAllDifferent()`.

Para construir as regiões, são inicialmente geradas todas as coordenadas da grelha. Estas coordenadas são depois baralhadas aleatoriamente e divididas em n^2 grupos, contendo cada grupo n^2 células.

Desta forma, garante-se que todas as posições da grelha são utilizadas exatamente uma vez e que nenhuma célula pertence a duas regiões diferentes.

Na função `sudoku_jigsaw`, as linhas e colunas continuam a ser representadas através de `path`. A principal diferença relativamente ao Sudoku normal é que os blocos `cube` deixam de ser adicionados e são substituídos pelas regiões geradas através de `regioes_jigsaw`.

As regiões, juntamente com as linhas, colunas e pistas, são posteriormente fornecidas à função `add_grupos`, mantendo a mesma lógica de resolução utilizada anteriormente.

```

In [ ]: # Apoio LLM: correção da geração das regiões Jigsaw e da validação
        específica das regiões

        # adiciona regiões

```

```

def regioes_jigsaw(n, dim):
    # existem  $n^2$  regiões (dim)
    # cada região tem  $n^2$  células sem repetições (dim)
    # nenhuma célula aparece em duas regiões e nenhuma fica por usar

    coordenadas = [] # lista de coordenadas
    for i in range(dim):
        for j in range(dim):
            coordenadas.append((i,j))

    # baralha as coordenadas
    random.shuffle(coordenadas)

    regioes = [] # lista das regiões

    for r in range(dim):
        regioao = box(n) # cria box para cada região

        inicio = r * dim
        fim = inicio + dim
        for c in coordenadas[inicio:inicio+dim]:
            i, j = c
            regioao.add(i,j)

        # adiciona a regioao ao grupo
        regioes.append(regiao)

    return regioes

# substituir cubes por regioes na função de sudoku
def sudoku_jigsaw(n, k=None): # k opcional para gerar pistas iniciais
    dim = n**2

    # criar modelo
    model, vars = modelo_sudoku(n)

    # criar listas de grupos
    grupos = []

    # adiciona todas as linhas com path
    for i in range(dim):
        # cada linha vai da coluna 0 até coluna dim-1
        linha = path(n, (i,0), (i,dim-1))
        grupos.append(linha)

    # adiciona todas as colunas com path
    for j in range(dim):
        # cada coluna vai da linha 0 até linha dim-1
        coluna = path(n, (0,j), (dim-1,j))
        grupos.append(coluna)

    # adiciona todos os blocos com regioao
    regioes = regioes_jigsaw(n, dim)
    for regioao in regioes:
        grupos.append(regiao)

```

```

# adiciona pistas aleatórias
pistas = pistas_aleatorias(n, k)
grupos.append(pistas)

# passar essa lista ao add_grupos
add_grupos(model, vars, grupos)

# resolver
res = resolve_sudoku(model, vars, n**2)

# imprimir solução
imprime_sudoku(res, n)

return res, pistas, regioes

# verifica se as regioes cumprem as restrições
def valida_regioes_jigsaw(matriz, regioes, lista_esperada, n):
    dim = n**2

    for regioao in regioes:
        valores = []

        for i, j in regioao.cells:
            valores.append(matriz[i][j])

        if sorted(valores) != lista_esperada:
            return False

    return True

# valida o sudoku jigsaw
def valida_solucao_jigsaw(matriz, pistas, regioes, n):
    dim = n**2

    # lista de valores que cada linha, coluna e bloco deve conter
    lista_esperada = list(range(1, dim+1))

    # validação das linhas
    if not valida_linhas(matriz, lista_esperada, dim):
        return False

    # validação das colunas
    if not valida_colunas(matriz, lista_esperada, dim):
        return False

    # validação dos blocos
    if not valida_regioes_jigsaw(matriz, regioes, lista_esperada, n):
        return False

    # validação das pistas
    if not valida_pistas(matriz, pistas):
        return False

```

```
return True
```

Teste sobre as funcionalidades de Jigsaw e a validação:

```
In [ ]: # Apoio LLM: geração de uma função para imprimir as coordenadas de cada
        região de forma mais legível

def imprime_regioes(regioes):
    for i, regioao in enumerate(regioes):
        print("Região", i, ":", list(regiao.cells.keys()))

def teste_jigsaw(n):
    matriz, pistas, regioes = sudoku_jigsaw(n)
    print()
    imprime_regioes(regioes)
    print()
    print("Solução válida: ", valida_solucao_jigsaw(matriz, pistas,
regioes, n))

teste_jigsaw(2)
print()
teste_jigsaw(3)
```

```

2 1 | 4 3
3 4 | 2 1
-----
4 3 | 1 2
1 2 | 3 4

```

Região 0 : [(2, 1), (0, 0), (0, 1), (3, 3)]
 Região 1 : [(1, 0), (0, 2), (2, 3), (1, 3)]
 Região 2 : [(2, 0), (3, 0), (1, 2), (3, 2)]
 Região 3 : [(1, 1), (2, 2), (0, 3), (3, 1)]

Solução válida: True

```

8 1 3 | 7 2 6 | 5 9 4
6 5 9 | 1 3 7 | 4 8 2
5 4 2 | 6 7 8 | 9 3 1
-----
9 2 7 | 5 6 1 | 3 4 8
3 8 6 | 4 5 2 | 1 7 9
1 9 4 | 3 8 5 | 2 6 7
-----
2 7 5 | 8 4 9 | 6 1 3
7 3 1 | 2 9 4 | 8 5 6
4 6 8 | 9 1 3 | 7 2 5

```

Região 0 : [(8, 7), (7, 1), (6, 1), (2, 8), (0, 0), (5, 7), (0, 8), (7, 4), (3, 3)]
 Região 1 : [(2, 5), (6, 5), (7, 8), (8, 5), (5, 8), (4, 5), (5, 0), (0, 6), (8, 0)]
 Região 2 : [(8, 2), (3, 4), (3, 0), (5, 5), (2, 1), (3, 5), (2, 4), (3, 1), (2, 7)]
 Região 3 : [(1, 7), (8, 6), (7, 3), (8, 3), (8, 1), (1, 4), (3, 7), (8, 4), (6, 2)]
 Região 4 : [(7, 0), (6, 4), (3, 6), (2, 6), (4, 2), (1, 8), (2, 0), (6, 7), (6, 3)]
 Região 5 : [(8, 8), (1, 0), (7, 6), (5, 2), (6, 0), (0, 7), (4, 7), (6, 8), (1, 3)]
 Região 6 : [(5, 4), (1, 1), (1, 5), (5, 1), (0, 5), (0, 1), (4, 3), (5, 6), (5, 3)]
 Região 7 : [(0, 4), (1, 6), (4, 6), (4, 0), (3, 8), (4, 8), (7, 7), (6, 6), (3, 2)]
 Região 8 : [(2, 2), (4, 4), (4, 1), (7, 2), (0, 3), (0, 2), (7, 5), (2, 3), (1, 2)]

Solução válida: True

Os testes realizados para $n = 2$ e $n = 3$ devolveram **True** na validação final, confirmando que as soluções respeitam as restrições das linhas, colunas, regiões e pistas.

Desta forma, verifica-se que a substituição dos blocos regulares por regiões arbitrárias funciona corretamente sem alterar o modelo CSP.

Escala

Nesta fase é realizado um teste com `n = 6`.

Neste caso, a dimensão da grelha é 36×36 , correspondendo a 1296 células e, consequentemente, 1296 variáveis inteiras no modelo CSP.

A construção do problema mantém exatamente a mesma estrutura utilizada para `n = 2` e `n = 3`: são criadas 36 linhas, 36 colunas e 36 blocos 6×6 , todos tratados através da mesma função responsável por aplicar as restrições `AddAllDifferent()`.

Para avaliar o desempenho, foi medido o tempo necessário para construir e resolver o modelo através de `time.perf_counter()`.

Após a resolução, a matriz resultante foi submetida à mesma função de validação utilizada nos testes anteriores, confirmando se cada linha, coluna e bloco contém exatamente os valores entre 1 e 36, sem repetições, e se as pistas foram mantidas.

```
In [ ]: # Apoio LLM: função sugerida para medir o tempo de execução

import time

def teste_escalas(n, k=None):
    inicio = time.perf_counter()

    matriz, pistas = sudoku(n,k)

    fim = time.perf_counter()

    print(f"Tempo de execução: {fim - inicio:.4f} segundos")

    if matriz is not None:
        print("Solução válida:", valida_solucao(matriz, pistas, n))

teste_escalas(6)
```

3 10 23 32 5 33 | 2 1 11 13 27 4 | 36 7 34 8 12 17 | 15 21 22 31 9 26 | 25 2
4 19 30 20 35 | 18 29 14 16 28 6
12 13 9 8 4 36 | 10 19 7 16 17 5 | 20 6 22 1 2 21 | 35 34 14 25 28 24 | 18 2
9 15 32 3 33 | 30 27 31 26 23 11
2 21 7 15 6 11 | 9 3 14 24 20 12 | 16 33 26 23 31 28 | 19 27 29 5 30 18 | 36
34 17 13 4 8 | 35 32 22 10 25 1
19 18 31 16 1 27 | 29 34 23 6 26 25 | 30 14 24 35 4 13 | 32 2 11 10 33 3 | 9
22 5 21 7 28 | 17 15 20 36 12 8
26 22 25 20 29 14 | 28 35 36 21 18 30 | 32 15 11 9 5 10 | 8 4 7 16 17 1 | 27
12 6 23 31 2 | 34 24 13 19 3 33
34 30 28 24 35 17 | 8 32 31 15 22 33 | 3 29 27 25 18 19 | 13 20 12 6 36 23 |
10 1 14 26 11 16 | 4 9 21 7 5 2

-
30 27 11 9 2 3 | 19 36 16 7 31 14 | 29 35 33 10 1 23 | 34 32 28 26 24 5 | 8
15 22 4 12 21 | 13 25 6 20 18 17
33 29 16 4 31 15 | 35 26 2 20 6 8 | 34 28 36 27 32 25 | 12 11 19 3 18 14 | 3
0 13 24 10 1 17 | 23 7 5 22 21 9
5 36 26 35 34 24 | 32 28 33 25 29 27 | 21 22 18 15 19 16 | 30 23 13 17 10 8
| 14 20 11 7 6 9 | 2 3 4 31 1 12
22 28 20 17 25 23 | 21 13 24 18 15 10 | 11 5 8 12 3 6 | 7 9 4 2 1 31 | 35 27
26 33 34 32 | 14 19 36 30 16 29
8 1 10 21 14 32 | 17 5 34 11 12 23 | 4 30 13 7 26 9 | 29 25 6 22 20 36 | 19
16 18 3 2 31 | 15 28 33 35 24 27
6 7 19 13 18 12 | 4 30 22 9 1 3 | 2 31 14 20 17 24 | 16 15 27 35 21 33 | 29
25 36 28 5 23 | 11 26 34 8 32 10

-
27 4 33 1 24 18 | 5 15 28 19 34 20 | 6 36 12 16 7 35 | 14 17 26 9 32 10 | 31
23 3 25 21 22 | 8 30 29 11 2 13
23 25 21 5 16 19 | 31 11 18 17 10 2 | 33 13 29 3 9 30 | 22 8 20 36 15 34 | 2
4 7 4 6 28 14 | 12 1 32 27 35 26
20 32 3 7 36 28 | 12 24 9 26 8 21 | 18 10 17 2 15 4 | 6 31 1 13 29 27 | 16 1
9 35 11 30 5 | 25 34 23 33 14 22
35 34 29 12 26 9 | 25 6 3 4 30 22 | 14 23 21 32 11 31 | 18 19 5 28 16 2 | 15
8 27 1 33 13 | 7 17 10 24 36 20
11 2 15 6 30 13 | 7 33 1 14 36 16 | 5 34 20 22 8 27 | 21 12 25 24 23 35 | 17
9 32 29 10 26 | 19 18 28 3 31 4
14 17 22 31 10 8 | 27 29 35 32 23 13 | 1 19 25 24 28 26 | 3 33 30 4 7 11 | 1
2 2 20 36 18 34 | 6 21 16 15 9 5

-
7 6 24 18 19 16 | 34 14 20 5 3 28 | 26 2 10 13 30 11 | 9 36 32 29 22 15 | 1
33 31 35 25 12 | 27 4 8 21 17 23
13 11 12 36 22 10 | 1 17 6 27 2 18 | 15 20 28 29 21 8 | 4 5 33 19 3 16 | 7 3
0 23 34 32 24 | 31 35 9 25 26 14
17 35 30 33 27 25 | 23 9 21 36 13 29 | 12 3 32 4 34 22 | 26 1 31 8 14 7 | 20
11 16 18 19 15 | 10 2 24 5 6 28
1 5 4 2 9 26 | 15 31 12 30 7 32 | 27 24 35 14 33 36 | 25 28 10 23 34 17 | 21
3 13 8 29 6 | 22 11 19 18 20 16
29 31 32 23 21 20 | 16 8 19 35 25 11 | 17 9 7 5 6 1 | 27 24 2 18 13 30 | 4 1
4 28 22 26 10 | 36 12 15 34 33 3
15 8 14 28 3 34 | 24 22 4 10 33 26 | 23 16 31 19 25 18 | 20 6 35 11 12 21 |
2 17 9 5 36 27 | 29 13 30 1 7 32

-

```

18 3 35 19 15 6 | 33 20 29 28 32 17 | 10 25 1 31 36 14 | 24 26 9 34 27 4 | 2
2 5 12 16 8 7 | 21 23 2 13 11 30
31 12 8 25 7 30 | 18 16 5 3 21 15 | 24 17 9 6 29 20 | 2 13 23 33 11 19 | 34
32 1 14 27 36 | 28 22 26 4 10 35
24 16 27 11 13 21 | 36 23 26 34 4 31 | 7 12 3 33 22 2 | 28 30 8 1 35 32 | 6
10 29 17 9 19 | 5 20 18 14 15 25
4 26 36 34 28 2 | 14 27 25 8 9 1 | 35 11 16 21 13 5 | 17 10 15 7 6 22 | 3 18
30 31 23 20 | 24 33 12 32 29 19
9 33 17 14 23 5 | 30 12 10 22 11 35 | 19 26 15 28 27 32 | 36 3 18 20 31 29 |
13 4 21 2 24 25 | 1 16 7 6 8 34
10 20 1 29 32 22 | 6 7 13 2 19 24 | 8 4 30 18 23 34 | 5 16 21 14 25 12 | 28
26 33 15 35 11 | 3 31 17 9 27 36
-----
-

```

```

16 19 18 10 12 31 | 26 25 17 23 35 6 | 13 21 4 34 20 7 | 1 22 24 32 2 28 | 1
1 36 8 9 14 3 | 33 5 27 29 30 15
21 23 13 26 11 1 | 20 18 15 12 16 19 | 25 8 6 17 14 29 | 31 35 36 30 5 9 | 3
3 28 7 27 22 4 | 32 10 3 2 34 24
25 24 5 3 8 7 | 13 2 30 31 14 9 | 28 27 23 26 35 33 | 11 18 34 12 4 20 | 32
6 10 19 15 29 | 16 36 1 17 22 21
36 15 6 22 33 4 | 11 21 8 1 5 34 | 9 32 19 30 24 3 | 10 29 16 27 26 25 | 23
31 2 12 17 18 | 20 14 35 28 13 7
28 14 2 27 17 29 | 22 10 32 33 24 36 | 31 1 5 11 16 15 | 23 7 3 21 19 6 | 26
35 34 20 13 30 | 9 8 25 12 4 18
32 9 34 30 20 35 | 3 4 27 29 28 7 | 22 18 2 36 10 12 | 33 14 17 15 8 13 | 5
21 25 24 16 1 | 26 6 11 23 19 31

```

Tempo de execução: 6.7912 segundos

Solução válida: True

Para avaliar o desempenho, foi medido o tempo necessário para construir e resolver o modelo através de `time.perf_counter()`. O tempo obtido foi de 6.83 segundos, aproximadamente.

O teste demonstra que a implementação é genérica e pode ser aplicada a valores de n superiores aos utilizados no Sudoku clássico. No entanto, o aumento de n provoca um crescimento significativo do número de variáveis e restrições, tornando a resolução mais exigente do ponto de vista computacional.

Uso de ferramentas LLM

Na realização dos desafios opcionais foi usada o ChatGPT como ferramenta de apoio à interpretação dos requisitos, definição de estratégias de resolução e revisão do código.

O apoio incidiu principalmente na implementação do X-Sudoku, nomeadamente na representação das diagonais como novos grupos de células, na validação dessas restrições e na correção de alguns erros encontrados durante os testes.

No bónus do Sudoku Jigsaw, foi utilizado apoio de LLM na correção da geração das regiões, na adaptação da montagem do Sudoku para substituir os blocos regulares por regiões arbitrárias e na definição da validação específica dessas regiões.

Foi também utilizado para explorar possíveis abordagens para outros bónus e para a realização do teste de escalabilidade com $n=6$, incluindo a medição do tempo de execução.

O diálogo utilizado durante a resolução encontra-se disponível em:

[<https://chatgpt.com/share/6ac55b45-de20-83eb-ade2-1ae8a31d6a7b>]

De forma geral, esta parte permitiu verificar que a estrutura genérica desenvolvida para o Sudoku clássico pode ser reutilizada e adaptada a outras variantes sem alterar a lógica principal do modelo.